

LAPORAN PRAKTIKUM STRUKTUR DATA

Double Linked List

Disusun Oleh :

Aliifah Felda Mufarrihati Salwaa

2511531011

Dosen Pengampu :

Dr. Wahyudi, M.T

Asisten Praktikum :

M Galid Avaro



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Laporan praktikum ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum Struktur Data yang membahas tentang Double Linked List. Melalui laporan ini penulis dapat memahami materi praktikum secara mendalam dan dapat lebih teliti, teratur, serta memiliki kemampuan yang baik dalam penulisan kode program sesuai kaidah akademik. Sehingga laporan ini dapat menjadi sarana belajar, dokumentasi kegiatan, dan referensi praktikum berikutnya.

Penulis menyadari bahwa laporan ini masih memiliki banyak kekurangan, baik dari isi maupun penyajiannya. Oleh karena itu, saran dan kritik sangat penulis harapkan guna untuk menyempurnakan laporan berikutnya.

Padang, 12 Mei 2026

Aliifah Felda Mufarrihati Salwaa

2511531011

DAFTAR ISI

KATA PENGANTAR.....	i
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang Praktikum	3
1.2 Tujuan Praktikum.....	3
1.3 Manfaat Praktikum.....	3
BAB II PEMBAHASAN	4
2.1. Dasar Teori.....	4
2.2. Class NodeDLL_2511531011.....	4
2.3. insertDLL_2511531011	5
2.4. PenelusuranDLL_2511531011	8
2.5. hapusDLL_2511531011.....	10
2.6. Tugas Pekan 5.....	13
BAB III KESIMPULAN.....	22
3.1. Kesimpulan	22
3.2. Saran	22
DAFTAR PUSTAKA.....	23

BAB I

PENDAHULUAN

1.1 Latar Belakang Praktikum

Setelah mempelajari berbagai implementasi ADT yakni array list, stack, queue, single linked list pada praktikum sebelumnya, kita juga harus mengetahui tentang penerapan ADT lainnya. Karena banyaknya implementasi ADT, kita harus dapat menguasai setiap implementasinya agar dapat membangun sebuah program efektif dan efisien, namun tetap sesuai dengan keinginan kita.

Oleh karena itu, pada praktikum kali ini kita akan mempelajari dan memahami tentang penerapan ADT lain yakni Double Linked List. Double Linked List merupakan struktur data dimana setiap elemen terhubung dengan elemen berikutnya dan terhubung juga dengan elemen sebelumnya. Setiap elemen (node) memiliki field yang berisi data dan field yang berisi pointer ke node berikutnya dan pointer ke node sebelumnya.

1.2 Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah sebagai berikut:

1. Memahami tentang implementasi ADT yakni Double Linked List.
2. Mengetahui kelebihan dan kekurangan Double Linked List dalam program.
3. Dapat membangun sebuah kode program dengan Double Linked List.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini adalah sebagai berikut:

1. Mampu mengimplementasikan Double Linked List dalam pembuatan program agar lebih efektif dan efisien.
2. Memahami penggunaan Double Linked List dalam program.

BAB II

PEMBAHASAN

2.1. Dasar Teori

Linked list merupakan struktur data linier dimana elemen-elemen yang disebut node disimpan secara berurutan tetapi tidak harus di lokasi memori yang berdekatan seperti array. Setiap node pada linked list memiliki 2 komponen utama, yakni: data (nilai node) dan pointer (penunjuk node berikutnya).

Linked list ini terbagi pada 2 jenis, yakni;

1. Single Linked List: setiap node hanya memiliki satu pointer menuju node berikutnya dan node terakhir menunjuk pada null
2. Double Linked List: setiap node memiliki dua pointer, yakni untuk menunjukkan node berikutnya dan untuk menunjukkan node sebelumnya.

Double linked list merupakan jenis dari linked list yang dimana setiap elemen terhubung dengan elemen setelahnya dan elemen sebelumnya. Menambahkan dan menghapus elemen lebih mudah dilakukan dalam double linked list karena kita tidak perlu melakukan *tracking* dari elemen sebelumnya saat traversal.

Penggunaan double linked list biasanya untuk sebuah program yang membutuhkan fungsi undo dan redo. Double linked list dapat mengimplementasikan struktur data yang populer digunakan seperti *binary tree*, *stack*, dan tabel hash.

2.2. Class NodeDLL_2511531011

```
package pekan6_2511531011;

public class NodeDLL_2511531011 {
    //mendefinisikan class node
    int data_1011; //data
    NodeDLL_2511531011 next_1011;
    NodeDLL_2511531011 prev_1011;

    //konstruktor
    public NodeDLL_2511531011 (int data_1011) {
        this.data_1011 = data_1011;
    }
}
```

```

        this.next_1011 = null;
        this.prev_1011 = null;
    }
}

```

Kode program 2.1: NodeDLL_2511531011

Class ini merupakan konstruktor untuk membuat double linked list. Disini kita menggunakan data_1011 untuk menunjukkan data yang tersimpan dan next_1011 sebagai pointer ke elemen berikutnya serta prev_1011 sebagai pointer ke elemen sebelumnya di set null agar tidak menunjuk kemanaapun.

2.3. insertDLL_2511531011

```

package pekan6_2511531011;

public class insertDLL_2511531011 {
    //menambahkan node di awal DLL
    static NodeDLL_2511531011 insertBegin_1011(NodeDLL_2511531011
head_1011, int data_1011) {
        //buat node baru
        NodeDLL_2511531011 new_node_1011 = new
NodeDLL_2511531011(data_1011);
        //jadikan pointer nextnya head
        new_node_1011.next_1011 = head_1011;
        //jadikan pointer prev head ke new_node
        if (head_1011 != null) {
            head_1011.prev_1011 = new_node_1011;
        }
        return new_node_1011;
    }

    //fungsi menambahkan node di akhir
    public static NodeDLL_2511531011
insertAtEnd_1011(NodeDLL_2511531011 head_1011, int new_data_1011) {
        //buat node baru
        NodeDLL_2511531011 newNode_1011 = new
NodeDLL_2511531011(new_data_1011);
        //jika DLL null jadikan head
        if (head_1011 == null) {
            head_1011 = newNode_1011;
        } else {
            NodeDLL_2511531011 curr_1011 = head_1011;
            while (curr_1011.next_1011 != null) {
                curr_1011 = curr_1011.next_1011;
            }
            curr_1011.next_1011 = newNode_1011;
            newNode_1011.prev_1011 = curr_1011;
        }
        return head_1011;
    }
}

```

```

    }

    //fungsi menambahkan node di posisi terakhir
    public static NodeDLL_2511531011 insertAtPosition_1011
(NodeDLL_2511531011 head_1011, int pos_1011, int new_data_1011) {
    //buat node baru
    NodeDLL_2511531011 new_node_1011 = new
NodeDLL_2511531011(new_data_1011);
    if (pos_1011 == 1) {
        new_node_1011.next_1011 = head_1011;
        if (head_1011 != null) {
            head_1011.prev_1011 = new_node_1011;
        }
        head_1011 = new_node_1011;
        return head_1011;
    }
    NodeDLL_2511531011 curr_1011 = head_1011;
    for (int i = 1; i < pos_1011 - 1 && curr_1011 != null; ++i)
    {
        curr_1011 = curr_1011.next_1011;
    }
    if (curr_1011 == null) {
        System.out.println("Posisi tidak ada");
        return head_1011;
    }
    new_node_1011.prev_1011 = curr_1011;
    new_node_1011.next_1011 = curr_1011.next_1011;
    curr_1011.next_1011 = new_node_1011;
    if (new_node_1011.next_1011 != null) {
        new_node_1011.next_1011.prev_1011 = new_node_1011;
    }
    return head_1011;
}

public static void printList_1011(NodeDLL_2511531011 head_1011) {
    NodeDLL_2511531011 curr_1011 = head_1011;
    while (curr_1011 != null) {
        System.out.print(curr_1011.data_1011 + " <-> ");
        curr_1011 = curr_1011.next_1011;
    }
    System.out.println();
}

public static void main(String[] args) {
    //membuat dll 2 <-> 3 <-> 5
    NodeDLL_2511531011 head_1011 = new NodeDLL_2511531011(2);
    head_1011.next_1011 = new NodeDLL_2511531011(3);
    head_1011.next_1011.prev_1011 = head_1011;
    head_1011.next_1011.next_1011 = new NodeDLL_2511531011(5);
    head_1011.next_1011.next_1011.prev_1011 = head_1011;

    //cetak DLL awal
    System.out.print("DLL awal: ");
    printList_1011(head_1011);

    //tambah 1 di awal
    head_1011 = insertBegin_1011(head_1011, 1);
}

```

```

        System.out.print("Simpul 1 ditambah di awal: ");
        printList_1011(head_1011);
        //tambah 6 di akhir
        System.out.print("Simpul 6 ditambah di akhir: ");
        int data_1011 = 6;
        head_1011 = insertAtEnd_1011(head_1011, data_1011);
        printList_1011(head_1011);
        //menambah 4 di posisi 4
        System.out.print("Tambah node 4 di posisi ke 4: ");
        int data2_1011 = 4;
        int pos_1011 = 4;
        head_1011 = insertAtPosition_1011(head_1011, pos_1011,
data2_1011);
        printList_1011(head_1011);
    }
}

```

Kode program 2.2: TambahSLL_2511531011

Pada kode program ini kita akan mengetahui dan mencoba bagaimana cara untuk membuat dan menambah data dalam struktur data double linked list. Disini kita menggunakan banyak method, yakni:

1. insertBegin_1011: method ini digunakan untuk menambahkan node pada bagian depan DLL dengan menjadikan node baru sebagai head dan head lama menjadi node setelah node baru.
2. insertAtEnd_1011: method ini digunakan untuk menambahkan node pada bagian akhir DLL. Menambahkan node dilakukan dengan mengecek apakah list kosong, jika iya maka node baru menjadi head. Jika tidak, lakukan loop untuk mencari node terakhir dan menambahkan node pada bagian belakang list, kemudian hubungkan dengan node sebelumnya.
3. insertAtPosition_1011: method ini digunakan untuk menyisipkan node baru pada posisi tertentu. Jika posisi target adalah 1, ganti head dengan node baru. Kemudian kita lakukan cek apakah posisi yang dimasukkan ada dalam list. Jika tidak valid, tampilkan "Posisi tidak ada" dan jika posisi valid, sisipkan node baru ke posisi yang dipilih.
4. printList_1011: method ini digunakan untuk mencetak seluruh data yang ada dalam list dengan melakukan loop hingga node terakhir.

Selain method yang digunakan, kita juga menggunakan class main untuk eksekusi langsung program. Pada class main ini kita akan memanggil class NodeDLL untuk

membuat node sesuai yang kita inginkan. Berikut adalah program yang akan kita lakukan pada class main:

1. Membuat double linked list baru dengan node 2 <-> 3 <-> 5. Disini kita harus menghubungkan node 2 dengan node 3 (next), node 3 dengan node 2 (prev) dan dengan node 5 (next), kemudian node 5 dengan node 3 (next).
2. Cetak list awal 2 <-> 3 <-> 5.
3. Menambah node berisi data 1 di awal dengan method insertBegin_1011 dan print list baru yang dihasilkan.
4. Menambah node berisi data 6 di akhir dengan method insertAtEnd_1011 dan print list baru yang dihasilkan.
5. Menambah node berisi data 4 di posisi ke 4 dengan method insertAtPosition_1011 dan print list baru yang dihasilkan.

Berikut adalah salah satu contoh output yang kita dapatkan dengan menggunakan banyak method insertDLL yang telah kita sediakan.

Output :

```
<terminated> insertDLL_2511531011 [Java Application] D:\Eclipse\eclipse\plugins\org.eclips
DLL awal: 2 <-> 3 <-> 5 <->
Simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
Simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
Tambah node 4 di posisi ke 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

2.4. PenelusuranDLL_2511531011

```
package pekan6_2511531011;

public class PenelusuranDLL_2511531011 {
    //fungsi penelusuran maju
    static void forwardTraversal_1011(NodeDLL_2511531011 head_1011) {
        //memulai penelusuran dari head
        NodeDLL_2511531011 curr_1011 = head_1011;
        //lanjutkan sampai akhir
        while (curr_1011 != null) {
            //print data
            System.out.print(curr_1011.data_1011 + " <-> ");
            //pindah ke node berikutnya
            curr_1011 = curr_1011.next_1011;
        }
        //print spasi
    }
}
```

```

        System.out.println();
    }

    //fungsi penelusuran mundur
    static void backwardTraversal_1011(NodeDLL_2511531011 tail_1011)
    {
        //mulai dari akhir
        NodeDLL_2511531011 curr_1011 = tail_1011;
        //lanjut sampai head
        while (curr_1011 != null) {
            //cetak data
            System.out.print(curr_1011.data_1011 + " <-> ");
            //pindah ke node sebelumnya
            curr_1011 = curr_1011.prev_1011;
        }
        //cetak spasi
        System.out.println();
    }

    public static void main(String[] args) {
        //cetak DLL
        NodeDLL_2511531011 head_1011 = new NodeDLL_2511531011(1);
        NodeDLL_2511531011 second_1011 = new NodeDLL_2511531011(2);
        NodeDLL_2511531011 third_1011 = new NodeDLL_2511531011(3);

        head_1011.next_1011 = second_1011;
        second_1011.prev_1011 = head_1011;
        second_1011.next_1011 = third_1011;
        third_1011.prev_1011 = second_1011;

        System.out.println("Penelusuran maju: ");
        forwardTraversal_1011(head_1011);

        System.out.println("Penelusuran mundur: ");
        backwardTraversal_1011(third_1011);
    }
}

```

Kode program 2.3: PenelusuranDLL_2511531011

Dalam kode program ini kita juga akan membuat beberapa method untuk membantu kita dalam menelusuri node. Berikut adalah method yang akan digunakan:

1. forwardTraversal_1011: untuk menelusuri dan mencetak list dari awal hingga akhir.
2. backwardTraversal_1011: untuk menelusuri dan mencetak list dari akhir hingga awal.

Pada class ini kita juga akan menambahkan class main yang berisi program:

1. Membuat DLL dengan node 1, 2, 3.

2. Menghubungkan node dengan node sebelumnya dan node setelahnya.
Contoh: node 1 dihubungkan dengan node 2 (next), node 2 dihubungkan dengan node 1 (prev) dan node 3 (next), terakhir node 3 dihubungkan dengan node 2 (prev).
3. Mencetak penelusuran maju dengan method forwardTraversal_1011.
4. Mencetak penelusuran mundur dengan method backwardTraversal_1011.

Output:

```
<terminated> PenelusuranDLL_2511531011 []
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->
```

2.5. hapusDLL_2511531011

```
package pekan6_2511531011;

public class hapusDLL_2511531011 {
    //fungsi menghapus node awal
    public static NodeDLL_2511531011 delHead_1011(NodeDLL_2511531011
head_1011) {
        if (head_1011 == null) {
            return null;
        }
        NodeDLL_2511531011 temp_1011 = head_1011;
        head_1011 = head_1011.next_1011;
        if (head_1011 != null) {
            head_1011.prev_1011 = null;
        }
        return head_1011;
    }

    //fungsi menghapus node akhir
    public static NodeDLL_2511531011
delLast_1011(NodeDLL_2511531011 head_1011) {
        if (head_1011 == null) {
            return null;
        }
        if (head_1011.next_1011 == null) {
            return null;
        }
        NodeDLL_2511531011 curr_1011 = head_1011;
        while (curr_1011.next_1011 != null) {
            curr_1011 = curr_1011.next_1011;
        }
    }
}
```

```

        //update pointer previous node
        if (curr_1011 != null) {
            curr_1011.prev_1011.next_1011 = null;
        }
        return head_1011;
    }

    //fungsi menghapus node di posisi tertentu
    public static NodeDLL_2511531011
delPos_1011(NodeDLL_2511531011 head_1011, int pos_1011) {
    //jika DLL kosong
    if (head_1011 == null) {
        return head_1011;
    }
    NodeDLL_2511531011 curr_1011 = head_1011;
    //telusuri sampai ke node yang akan dihapus
    for (int i = 1; curr_1011 != null && i < pos_1011;
++i) {
        curr_1011 = curr_1011.next_1011;
    }
    //jika posisi tidak ditemukan
    if (curr_1011 == null) {
        return head_1011;
    }
    //update pointer
    if (curr_1011.prev_1011 != null) {
        curr_1011.prev_1011.next_1011 =
curr_1011.next_1011;
    }
    if (curr_1011.next_1011 != null) {
        curr_1011.next_1011.prev_1011 =
curr_1011.prev_1011;
    }
    //jika yang dihapus head
    if (head_1011 == curr_1011) {
        head_1011 = curr_1011.next_1011;
    }
    return head_1011;
}

//fungsi mencetak DLL
public static void printList_1011(NodeDLL_2511531011
head_1011) {
    NodeDLL_2511531011 curr_1011 = head_1011;
    while (curr_1011 != null) {
        System.out.print(curr_1011.data_1011 + " ");
        curr_1011 = curr_1011.next_1011;
    }
    System.out.println();
}

public static void main(String[] args) {
    //buat sebuah DLL
    NodeDLL_2511531011 head_1011 = new NodeDLL_2511531011(1);
    head_1011.next_1011 = new NodeDLL_2511531011(2);
    head_1011.next_1011.prev_1011 = head_1011;
    head_1011.next_1011.next_1011 = new NodeDLL_2511531011(3);
}

```

```

        head_1011.next_1011.next_1011.prev_1011 =
head_1011.next_1011;
        head_1011.next_1011.next_1011.next_1011 = new
NodeDLL_2511531011(4);
        head_1011.next_1011.next_1011.next_1011.prev_1011 =
head_1011.next_1011.next_1011;
        head_1011.next_1011.next_1011.next_1011.next_1011 = new
NodeDLL_2511531011(5);
        head_1011.next_1011.next_1011.next_1011.next_1011.prev_1011
= head_1011.next_1011.next_1011.next_1011;

        System.out.print("DLL awal: ");
        printList_1011(head_1011);

        System.out.print("Setelah head dihapus: ");
        head_1011 = delHead_1011(head_1011);
        printList_1011(head_1011);

        System.out.print("Setelah node terakhir dihapus: ");
        head_1011 = delLast_1011(head_1011);
        printList_1011(head_1011);

        System.out.print("Menghapus node ke 2: ");
        head_1011 = delPos_1011(head_1011, 2);

        printList_1011(head_1011);
    }
}

```

Kode program 2.4: hapusDLL_2511531011

Dalam kode program ini kita juga akan membuat beberapa method untuk membantu kita dalam menghapus sebuah node. Berikut adalah method yang akan digunakan:

1. delHead_1011: untuk menghapus node pertama dan mengubah nilai head dengan node setelahnya.
2. delLast_1011: untuk menghapus node terakhir dengan cara mencari node terakhir, menghapus node terakhir, dan mengubah node terakhir dengan node sebelumnya.
3. delPos_1011: untuk menghapus node pada indeks tertentu. Jika tujuan adalah node 1, maka hapus dan jadikan node berikutnya head. Jika selain node pertama, telusuri node dan hapus.
4. printList_2511531011: untuk print seluruh node yang ada pada list.

Disini kita juga memiliki class main untuk menjalankan program menggunakan seluruh method yang telah kita gunakan. Berikut adalah beberapa program yang akan kita lakukan:

1. Membuat node baru dengan data 1, 2, 3, 4, 5
2. Print DLL awal dengan printList_1011
3. Menghapus head dengan method delHead_1011 dan print list.
4. Menghapus node terakhir dengan method delLast_1011 dan print list.
5. Menghapus node ke 2 dengan method delPos_1011 dan print list.

Output :

```
<terminated> hapusDLL_2511531011 [Java Application] D
DLL awal: 1 2 3 4 5
Setelah head dihapus: 2 3 4 5
Setelah node terakhir dihapus: 2 3 4
Menghapus node ke 2: 2 4
```

2.6. Tugas Pekan 5

A. Lagu_2511531011

```
package pekan6_2511531011;

public class Lagu_2511531011 {
    //mendefinisikan class node
    String judul_1011;
    String penyanyi_1011;
    Lagu_2511531011 next_1011;
    Lagu_2511531011 prev_1011;

    //konstruktor
    public Lagu_2511531011 (String judul_1011, String
    penyanyi_1011) {
        this.judul_1011 = judul_1011;
        this.penyanyi_1011 = penyanyi_1011;
        this.next_1011 = null;
        this.prev_1011 = null;
    }

    //method getter
    public String getJudul_1011() { return judul_1011; } //get
    judul
    public String getPenyanyi_1011() { return penyanyi_1011; }
    //get penyanyi
```

```

        public Lagu_2511531011 getPointer_1011() { return
next_1011; } //get pointer

        //method setter
        public void setJudul_1011(String judul_1011) {
this.judul_1011 = judul_1011; } //set judul
        public void setPenyanyi_1011(String penyanyi_1011) {
this.penyanyi_1011 = penyanyi_1011; } //set penyanyi
        public void setPointer_1011(Lagu_2511531011 next_1011) {
this.next_1011 = next_1011; } //set pointer

        //untuk cetak data lagu
        @Override
        public String toString () {
            return "Judul Lagu: " + judul_1011 + ", Penyanyi: "
+ penyanyi_1011;
        }
    }
}

```

Kode Program 2.5: Lagu_2511531011

Pada kode program ini kita akan membuat konstruktor untuk double linked list dengan atribut judul, penyanyi, pointer ke belakang (prev) dan pointer ke depan (next). Disini kita juga membuat method setter dan getter untuk setiap atribut yang ada. Kita juga menggunakan override untuk mencetak data lagu.

B. Musik_2511531011

```

package pekan6_2511531011;

import java.util.Scanner;

public class Musik_2511531011 {
    private Lagu_2511531011 head_1011;
    public static Lagu_2511531011 tambahLagu_1011(Lagu_2511531011
head_1011, String newJudul_1011, String newPenyanyi_1011) {
        //buat node baru
        Lagu_2511531011 newNode_1011 = new
Lagu_2511531011(newJudul_1011, newPenyanyi_1011);
        //jika DLL null jadikan head
        if (head_1011 == null) {
            head_1011 = newNode_1011;
        } else {
            Lagu_2511531011 curr_1011 = head_1011;
            while (curr_1011.next_1011 != null) {
                curr_1011 = curr_1011.next_1011;
            }
            curr_1011.next_1011 = newNode_1011;
            newNode_1011.prev_1011 = curr_1011;
        }
    }
}

```

```

        System.out.println("Lagu " + newJudul_1011 + " - " +
newPenyanyi_1011 + " berhasil dimasukkan!");
        return head_1011;
    }

    //fungsi menghapus node awal
    public static Lagu_2511531011
hapusLaguAwal_1011(Lagu_2511531011 head_1011) {
        if (head_1011 == null) {
            System.out.println("Playlist kosong.");
            return null;
        }
        Lagu_2511531011 temp_1011 = head_1011;
        head_1011 = head_1011.next_1011;
        if (head_1011 != null) {
            head_1011.prev_1011 = null;
        }
        System.out.println("Lagu " + temp_1011.judul_1011 +
" - " + temp_1011.penyanyi_1011 + " berhasil dihapus!");
        return head_1011;
    }

    //fungsi menampilkan semua lagu dari awal ke akhir
    static void tampilMaju_1011(Lagu_2511531011 head_1011) {
        if (head_1011 == null) {
            System.out.println("Playlist kosong.");
            return;
        }

        System.out.println("Urutan lagu dari awal sampai
akhir:");

        //memulai penelusuran dari head
        Lagu_2511531011 curr_1011 = head_1011;
        //lanjutkan sampai akhir
        while (curr_1011 != null) {
            //print data
            System.out.print("Lagu: " +
curr_1011.judul_1011 + ", Penyanyi: " + curr_1011.penyanyi_1011 + " <->
\n");

            //pindah ke node berikutnya
            curr_1011 = curr_1011.next_1011;
        }
        //print spasi
        System.out.println();
    }

    //fungsi menampilkan lagu dari akhir sampai awal
    static void tampilMundur_1011(Lagu_2511531011 tail_1011) {
        if (tail_1011 == null) {
            System.out.println("Playlist kosong.");
            return;
        }

        System.out.println("Urutan lagu dari akhir sampai
awal:");

        //mulai dari akhir
        Lagu_2511531011 curr_1011 = tail_1011;

```



```

        //lanjut sampai head
        while (curr_1011 != null) {
            //cetak data
            System.out.print("Lagu: " +
curr_1011.judul_1011 + ", Penyanyi: " + curr_1011.penyanyi_1011 + " <->
\n");

            //pindah ke node sebelumnya
            curr_1011 = curr_1011.prev_1011;
        }
        //cetak spasi
        System.out.println();
    }

    //fungsi cari lagu berdasarkan judul
    public void cariLagu_1011(String cariJudul_1011) {

        // cek apakah list kosong
        if (head_1011 == null) {

            System.out.println("Playlist kosong.");
            return;
        }

        Lagu_2511531011 curr_1011 = head_1011;
        int posisi_1011 = 1;
        boolean ditemukan_1011 = false;
        // traversal untuk pencarian data
        while (curr_1011 != null) {

            // pencarian tanpa membedakan huruf besar
            kecil
                if
(curr_1011.getJudul_1011().equalsIgnoreCase(cariJudul_1011)) {
                    System.out.println("Lagu ditemukan pada
posisi " + posisi_1011);

                    System.out.println(curr_1011);
                    ditemukan_1011 = true;
                    break;
                }
                curr_1011 = curr_1011.getPointer_1011();
                posisi_1011++;
            }
            // jika data tidak ditemukan
            if (!ditemukan_1011) {
                System.out.println("lagu dengan judul " +
cariJudul_1011 + " tidak ditemukan.");
            }
        }

        // method untuk menampilkan menu
        public static void tampilkanMenu_1011() {
            System.out.println("\n=== Playlist Musik NIM:
2511531011 ===");
            System.out.println("1. Tambah Lagu");
            System.out.println("2. Hapus Lagu Pertama");
            System.out.println("3. Lihat Playlist (Maju)");
            System.out.println("4. Lihat Playlist (Mundur)");
        }
    }

```

```

        System.out.println("5. Cari lagu");
        System.out.println("6. Keluar");
        System.out.print("Pilihan : ");
    }

    public static void main(String[] args) {
        Scanner input_1011 = new Scanner(System.in);
        Musik_2511531011 m_1011 = new Musik_2511531011();
        int pilihan_1011;
        do {
            tampilkanMenu_1011();
            pilihan_1011 = input_1011.nextInt();
            input_1011.nextLine();
            switch (pilihan_1011) {
                case 1:
                    System.out.print("Masukkan judul
lagu : ");
                    String judul_1011 =
input_1011.nextLine();
                    System.out.print("Masukkan
penyanyi : ");
                    String penyanyi_1011 =
input_1011.nextLine();
                    m_1011.head_1011 =
tambahLagu_1011(m_1011.head_1011, judul_1011, penyanyi_1011);
                    break;
                case 2:
                    m_1011.head_1011 =
hapusLaguAwal_1011(m_1011.head_1011);
                    break;
                case 3:
                    tampilMaju_1011(m_1011.head_1011);
                    break;
                case 4:
                    Lagu_2511531011 tail_1011 =
m_1011.head_1011;
                    while (tail_1011 != null &&
tail_1011.next_1011 != null) {
                        tail_1011 =
tail_1011.next_1011;
                    }
                    tampilMundur_1011(tail_1011);
                    break;
                case 5:
                    System.out.print("Masukkan judul
lagu yang dicari : ");
                    String cariJudul_1011 =
input_1011.nextLine();
                    m_1011.cariLagu_1011(cariJudul_1011);
                    break;
                case 6:
                    System.out.println("Keluar dari
program.");
            }
        }
    }
}

```

```

                                break;
                                default:
                                System.out.println("Pilihan
tidak valid.");
                                }
                                } while (pilihan_1011 != 6);
                                input_1011.close();
                                }
}

```

Kode program 2.6: Musik_2511531011

Pada kode program ini kita akan membuat beberapa method untuk kita panggil pada menu main. Beberapa method yang digunakan adalah:

1. tambahLagu_1011: disini kita membuat node baru dengan memanggil Lagu_2511531011(newJudul_1011, newPenyanyi_1011). Kita akan menjadikan node baru sebagai head jika sebelumnya DLL adalah null. Jika DLL tidak null, kita akan melakukan pencarian dari node awal hingga node akhir dan menambahkan node di bagian paling ujung ditambah dengan pointer ke depan dan ke belakang. Setelah itu kita akan melakukan print keberhasilan judul lagu dan penyanyi yang telah dimasukkan.
2. hapusLaguAwal_1011: pada method ini pertama kali kita akan cek apakah DLL null, jika iya maka print playlist kosong. Jika tidak, kita akan menghapus node awal (head) kemudian mengubah head menjadi node setelahnya, kemudian print lagu yang sudah dihapus.
3. tampilMaju_1011: pada method ini pertama kali kita akan cek apakah DLL null, jika iya maka print playlist kosong. Jika tidak, kita akan melakukan penelusuran lagu dari awal hingga akhir dan print semuanya.
4. tampilMundur_1011: pada method ini pertama kali kita akan cek apakah DLL kosong, jika iya maka print playlist kosong. Jika tidak, kita akan melakukan penelusuran DLL dari akhir hingga awal kemudian mencetak semua playlist.
5. cariLagu_1011: pada method ini pertama kali kita akan cek apakah DLL null, jika iya maka print playlist kosong. Jika tidak, kita akan melakukan pencarian dengan menelusuri DLL dan menggunakan equalsIgnoreCase agar pencarian bukan case sensitif.

6. tampilkanMenu_1011: pada method ini kita akan membuat beberapa menu untuk output nantinya.

Setelah membuat method yang dibutuhkan, kemudian kita akan membuaat class main agar program dapat dijalankan. Berikut adalah program yang akan dieksekusi:

1. Membuat scanner input_1011 agar user dapat input data.
2. Memanggil class Musik_1011 sebagai m_1011 dan membuat variabel pilihan_1011.
3. Menggunakan perulangan do while yang berisik siwtch case, dimana:
 - a. Switch pilihan_1011
 - b. Case 1: meminta input dari user untuk memasukkan judul lagu dan penyanyi dengan method tambahLagu_1011.
 - c. Case 2: menghapus lagu bagian awal dengan memanggil method hapusLaguAwal_1011.
 - d. Case 3: print seluruh lagu dalam playlist dengan memanggil method tampilMaju_1011
 - e. Case 4: print seluruh lagu dalam playlist dengan memanggil method tampilMundur_1011
 - f. Case 5: mencari judul lagu sesuai dengan permintaan user dengan memannnggil method cariLagu_1011.
 - g. Case 6: keluar dari program
 - h. Default: pilihan tidak valid.
4. Pemilihan case ini akan terus berulang jika kita memilih pilihan 1-5, dan pilihan 6 untuk keluar.

Berikut adalah salah satu contoh output yang dapat terjadi jika kita menggunakan program ini:

```

=== Playlist Musik NIM: 2511531011 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari lagu
6. Keluar
Pilihan : 1
Masukkan judul lagu : drowning
Masukkan penyanyi : woodz
Lagu drowning - woodz berhasil dimasukkan!

=== Playlist Musik NIM: 2511531011 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari lagu
6. Keluar
Pilihan : 1
Masukkan judul lagu : drop dead
Masukkan penyanyi : olivia rodrigo
Lagu drop dead - olivia rodrigo berhasil dimasukkan!

=== Playlist Musik NIM: 2511531011 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari lagu
6. Keluar
Pilihan : 1
Masukkan judul lagu : adore u
Masukkan penyanyi : seventeen
Lagu adore u - seventeen berhasil dimasukkan!

=== Playlist Musik NIM: 2511531011 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari lagu
6. Keluar
Pilihan : 3
Urutan lagu dari awal sampai akhir:
Lagu: drowning, Penyanyi: woodz <->
Lagu: drop dead, Penyanyi: olivia rodrigo <->
Lagu: adore u, Penyanyi: seventeen <->

=== Playlist Musik NIM: 2511531011 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari lagu
6. Keluar
Pilihan : 4
Urutan lagu dari akhir sampai awal:
Lagu: adore u, Penyanyi: seventeen <->
Lagu: drop dead, Penyanyi: olivia rodrigo <->
Lagu: drowning, Penyanyi: woodz <->

=== Playlist Musik NIM: 2511531011 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari lagu
6. Keluar
Pilihan : 2

```

<pre> Pilihan : 2 Lagu drowning - woodz berhasil dihapus! === Playlist Musik NIM: 2511531011 === 1. Tambah Lagu 2. Hapus Lagu Pertama 3. Lihat Playlist (Maju) 4. Lihat Playlist (Mundur) 5. Cari lagu 6. Keluar Pilihan : 3 Urutan lagu dari awal sampai akhir: Lagu: drop dead, Penyanyi: olivia rodrigo <-> Lagu: adore u, Penyanyi: seventeen <-> === Playlist Musik NIM: 2511531011 === 1. Tambah Lagu 2. Hapus Lagu Pertama 3. Lihat Playlist (Maju) 4. Lihat Playlist (Mundur) 5. Cari lagu 6. Keluar Pilihan : 44 Pilihan tidak valid. === Playlist Musik NIM: 2511531011 === 1. Tambah Lagu 2. Hapus Lagu Pertama 3. Lihat Playlist (Maju) 4. Lihat Playlist (Mundur) 5. Cari lagu 6. Keluar Pilihan : 4 Urutan lagu dari akhir sampai awal: Lagu: adore u, Penyanyi: seventeen <-> Lagu: drop dead, Penyanyi: olivia rodrigo <-> </pre>	<pre> Pilihan : 4 Urutan lagu dari akhir sampai awal: Lagu: adore u, Penyanyi: seventeen <-> Lagu: drop dead, Penyanyi: olivia rodrigo <-> === Playlist Musik NIM: 2511531011 === 1. Tambah Lagu 2. Hapus Lagu Pertama 3. Lihat Playlist (Maju) 4. Lihat Playlist (Mundur) 5. Cari lagu 6. Keluar Pilihan : 5 Masukkan judul lagu yang dicari : adore u Lagu ditemukan pada posisi 2 Judul Lagu: adore u, Penyanyi: seventeen === Playlist Musik NIM: 2511531011 === 1. Tambah Lagu 2. Hapus Lagu Pertama 3. Lihat Playlist (Maju) 4. Lihat Playlist (Mundur) 5. Cari lagu 6. Keluar Pilihan : 2 Lagu drop dead - olivia rodrigo berhasil dihapus! === Playlist Musik NIM: 2511531011 === 1. Tambah Lagu 2. Hapus Lagu Pertama 3. Lihat Playlist (Maju) 4. Lihat Playlist (Mundur) 5. Cari lagu 6. Keluar Pilihan : 2 Lagu adore u - seventeen berhasil dihapus! </pre>
--	--

```

Lagu adore u - seventeen berhasil dihapus!

=== Playlist Musik NIM: 2511531011 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari lagu
6. Keluar
Pilihan : 33
Pilihan tidak valid.

=== Playlist Musik NIM: 2511531011 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari lagu
6. Keluar
Pilihan : 3
Playlist kosong.

=== Playlist Musik NIM: 2511531011 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari lagu
6. Keluar
Pilihan : 4
Playlist kosong.

=== Playlist Musik NIM: 2511531011 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)

```

```

=== Playlist Musik NIM: 2511531011 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari lagu
6. Keluar
Pilihan : 5
Masukkan judul lagu yang dicari : adore u
Playlist kosong.

=== Playlist Musik NIM: 2511531011 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari lagu
6. Keluar
Pilihan : 6
Keluar dari program.

```

BAB III

KESIMPULAN

3.1. Kesimpulan

Penggunaan struktur data double linked list lebih memudahkan kita dalam memasukkan dan menghapus sebuah node dari list. Double linked list juga memungkinkan kita untuk mencari atau menelusuri node dari belakang sehingga tidak membutuhkan waktu yang lama untuk akses node bagian belakang. Penggunaan linked list ini dapat disesuaikan dengan kebutuhan program apakah membutuhkan single linked list atau double linked list.

3.2. Saran

Karena banyaknya implementasi dari ADT, maka mahasiswa sebaiknya terus mengulang dan mempelajari setiap implementasi ADT. Mahasiswa kini hendaknya sudah mampu untuk memahami dan menggunakan struktur data double linked list. Untuk lebih mengasah keterampilan demi mendapatkan hasil program yang baik, hendaknya mahasiswa mampu dan terus belajar, salah satunya implementasi ADT.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, dan C. Stein, *Introduction to Algorithms*, 3rd ed., Cambridge, MA: MIT Press, 2009.
- [2] Wahyudi, *Struktur Data dan Algoritma*, Padang: Informatika, 2026
- [3] M.T. Goodrich, *Data Structures and Algorithms in Java*
- [4] “Linked List,” *Baeldung* [Daring}. Tersedia pada: <https://www.baeldung.com/cs/linked-list-data-structure> [Diakses: 13-Mei-2026]